TP 12 : Créer et manipuler des fils d'exécution

Ce TP est à réaliser en C.

Préliminaires

Ce TP a pour but de vous familiariser avec les fils d'exécution. On cherchera donc à ne pas créer des programmes trop compliqués. Il est un peu long mais est vraiment progressif.

Si vous avez des questions de syntaxe, essayez de réaliser une recherche via man. N'hésitez pas à m'appeler si ce n'est pas suffisant, je vous éclaircirai avec plaisir.

On cherche ici à répartir différents fils d'exécution pour calculer la nombre d'éléments pairs d'un tableau. Dans un premier temps, on implémente un algorithme très simple pour se familiariser avec la syntaxe.

Un premier programme

On cherche ici simplement à créer de multiples fils d'exécution qui affichent le début de leur exécution, attendent puis affichent la fin de leur exécution.

Question 1 : Écrire une fonction `void* affichage(void* id)` qui, étant donné un identifiant entier, affiche le début de son exécution avec son identifiant, attend un nombre de secondes aléatoire entre 0 et 10 puis affichera la fin de l'exécution.

On pourra utiliser les fonction `void sleep(int duree)` qui, étant donnée une durée, n'effectue rien pendant le temps spécifié par durée en secondes. Il faut importer la bibliothèque unistd pour utiliser cette fonction. On pourra aussi utiliser la fonction `int rand()` qui génère un nombre pseudo-aléatoire entre 0 et `RAND_MAX`.

Question 2 : Écrire une fonction `pthread_t* creer_tableau(int taille)` qui, étant donné un entier, alloue la mémoire nécessaire pour manipuler autant d'objets de type `pthread_t`.

Question 3 : Écrire une fonction `void paralleliser(int p, ptread_t* tableau_fils)` qui, étant donné un tableau de fils d'exécution et sa taille, crée un fil par élément du tableau en l'exécutant sur la fonction `affichage` avec un identifiant différent des précédents.

Question 4 : Compléter la fonction `main` pour qu'après compilation, le programme généré demande un nombre de fils à créer puis exécute `paralleliser` sur autant d'entre eux. On n'oubliera pas de réaliser un `pthread_join` à la fin de l'exécution.

Compter les pairs

Dans cette parie, on cherche à compter le nombre d'éléments pairs dans un tableau d'entiers. Pour cela, on utilise un tableau `T`, variable globale. On l'initialise à l'aide de la fonction `initialiser` suivante :

```
int* initialiser(int n)
{
    int* t = malloc(sizeof(int)*n);
    for (int i = 0; i < n; i++)
    {
        t[i] = rand()%n;
    }
    return t;
}
```

De plus, on a accès à un entier `res`, dans lequel on veut stocker le nombre calculé.

Question 5 : Écrire une fonction `int sous_nombre(int debut, int taille)` qui, renvoie le nombre d'entiers pairs entre les indices `debut` inclus et `debut+taille` exclus.

Question 6 : Proposer un type pour pouvoir fournir à chaque fil le début et la taille du sous-tableau qu'il doit traiter.

Question 7 : Écrire une fonction `int calculer(int k,int n)` qui, étant donnés un nombre de fils d'exécution k et la taille n du tableau, renvoie le nombre d'entiers pairs dans le tableau. On découpera le tableau en k portions que l'on confiera à chaque fil.

Question 8 : Votre solution garantit-elle l'exclusion mutuelle ?

Question 9 : Pour $n = 10000$, calculer la valeur de k pour laquelle `calculer` s'exécute le plus rapidement.

Simulation d'un mini serveur

On simule maintenant l'exécution d'un serveur recevant des requêtes. Celles-ci sont sont définies par leur durée d'exécution pendant laquelle les fils s'endorment. On traite les requêtes de manière similaire aux fonctions qu'on a utilisé la semaine dernière. Pour l'instant, en l'absence de concurrence, il ne traite qu'une requête à la fois.

Un squelette de code est déjà disponible dans le fichier `TP12.c`.

Avant tout, voici quelques rappels sur la gestion des sémaphores en C. Ils sont définis avec l'en-tête `semaphore.h`. Un sémaphore est un objet de type `sem_t`. En pratique, un sémaphore est un entier positif qui propose deux opérations :

- Incrémenter le sémaphore à l'aide de `int sem_post(sem_t *sem)`.
- Attendre que le sémaphore ait une valeur non nulle puis la décrémenter à l'aide de `int sem_wait(sem_t *sem)`. Le fil d'exécution est bloqué jusqu'à ce que la décrémentation soit possible.

Pour créer un sémaphore, on utilise la fonction `int sem_inti(sem_t *sem, int pshared, unsigned int value)`. Le second paramètre ne nous intéresse pas et sera fixé à `NULL`. Le dernier paramètre correspond à la valeur initiale du sémaphore. Pour détruire un sémaphore, on utilise la fonction `int sem_destroy(sem_t *sem)`

Introduire la concurrence

Question 10 : Modifier le code pour qu'un nouveau thread soit créé via `pthread_create` pour traiter chaque requête.

Question 11 : On souhaite limiter le nombre de connexions simultanées au serveur afin d'éviter de le surcharger si de nombreuses requêtes arrivent en même temps. Utiliser un sémaphore pour limiter le nombre de fils simultanés à 4. Comme précédemment, il n'est pas nécessaire de modifier le code des fonctions `attendreClient` et `executerRequete`.

Question 12 : La création et la destruction d'un fil d'exécution à chaque requête peut être coûteux. Notre nouvel objectif est de créer les 4 fils au début du programme, puis de communiquer les requêtes entrantes au premier fil inactif. À cet effet, on pourra définir les variables globales suivantes :

```
sem_t semRequete; // semaphore binaire qui permet aux threads d'attendre
    que le thread principal leur envoie une requete.
sem_t semTraitement; // semaphore binaire qui permet au thread principal d
    'attendre qu'un thread de travail ait pris en compte la derniere
    requete en attente.
requete *reqAttente; // requete en attente de traitement par un des
    threads de travail
```

On pourra aussi faire le choix d'utiliser des mutex. On rappelle que les variables globales sont crées à l'extérieur du code des fonctions mais doivent être initialisées dans celles-ci.

Stocker les requêtes en attente

Quand nos 4 threads sont occupés, le thread principal est bloqué et les éventuelles nouvelles requêtes sont ignorées. Pour corriger ce problème, il est possible de maintenir une file de requêtes, qui peut prendre la forme d'une liste simplement chaînée ou d'un tableau, au lieu de conserver une unique requête en attente. Le thread principal ajoute les requêtes entrantes en queue de file, et les threads de travail consomment les requêtes en début de file. C'est le principe du problème *producteur-consommateur*.

On utilise la structure de donnée suivante pour modéliser une file de requêtes :

```
typedef struct file_element_t{
  struct file_element_t* next;
  requete* req;
} file_element;


typedef struct file_t{
  file_element* tete;
  file_element* queue;
} file;
```

Question 13 : Écrire des fonctions `bool vide(file* f)` et `void inserer(file* f, requete* r)` qui permettent de tester si la file est vide et d'y insérer un élément.

Question 14 : Écrire une fonction `requete* dequeue(file* f)` qui renvoie la requête en tête de la file et modifie celle-ci.

Question 15 : À l'aide d'un sémaphore ou d'un mutex et de la structure que vous venez de contruire, répondre au problème étudié.